

Native Word-Level Macro Evaluation in a GPU RTL Simulator

Deliverable E: Scheduling Equations, Memory Architecture, Occupancy Analysis and Verification Methodology

Takneek Zenith — The Big-GEM Theory

Abstract

We extend NVIDIA’s GEM boomerang-scheduled GPU logic simulator so that DSP48E2, CARRY4 and SRLC32E primitives survive Yosys synthesis as word-level cells and are evaluated natively on the GPU ALU, instead of being shredded into 1-bit AND-inverter logic. This document gives the modified levelization equations that make a heterogeneous DAG schedulable, the VRAM layout that keeps macro operands 64-bit aligned and coalesced, a measured occupancy and register profile on `sm.86`, an alignment invariant that variable-length instruction sections must preserve, and a layered verification argument in which each oracle’s authority is bounded explicitly rather than assumed.

1 The Invariant That Macros Break

Stock GEM rests on a single structural invariant:

Every value inside a partition is produced by the AND-gate cover, and reaches its consumers through the boomerang hierarchy.

Word-level macros violate it in three independent ways. A macro output is produced by a separate evaluation phase, written into a separate buffer, on a separate schedule. Each consequence had to be addressed explicitly, and the equations in Section 2 exist precisely to restore a well-defined order once the invariant no longer holds.

Concretely, in the kernel the boomerang stages of one partition communicate through `shared_state`, while the macro evaluation phase writes into `shared_writeouts`. These are different buffers. A macro result therefore cannot become visible to boolean logic within the same partition at all — not in a later boomerang stage, not under any batching order. The only ordering primitive strong enough is the cooperative grid synchronisation between *major* stages, and the scheduling equations are built around that fact.

2 Modified DAG Levelization Equations

2.1 Notation

Let $G = (V, E)$ be the AND-inverter graph after macro-preserving synthesis. Each $v \in V$ is an *aigpin*. Write:

- $\text{fanin}(v)$ — the AND-gate fan-in of v (empty for non-AND nodes);
- $\mathcal{M}$ — the set of macro instances;
- $\text{combin}(m)$ — the *combinational* input pins of macro m (CARRY4: $S, DI, CIN, CYINIT$; SRLC32E: $A[4:0]$; DSP48E2: $\emptyset$);
- $\text{seqin}(m)$ — *all* pins macro m latches on the clock edge, including its clock enable;

- $\mathcal{O}_c \subset V$ — aigpins driven by a *combinational* macro output (CARRY4 CO, O ; SRLC32E Q, Q_{31});
- $\mathcal{O}_s \subset V$ — aigpins driven by a *clocked* macro output (DSP48E2 P).

We define two integer labels per node. They coincide everywhere except on $\mathcal{O}_c$, and that single point of divergence is the entire content of the heterogeneous schedule.

$\text{avail}(v)$ the major stage in which v 's value may be *read* by boolean logic;

$\text{eval}(v)$ the major stage in which the producer of v is *executed*.

2.2 The equations

$$\text{avail}(v) = 0, \quad \begin{array}{l} v \text{ a primary input, DFF } Q, \text{ SRAM read data,} \\ \text{or } v \in \mathcal{O}_s \end{array} \quad (1)$$

$$\text{avail}(v) = \max_{u \in \text{fanin}(v)} \text{avail}(u), \quad v \text{ an AND gate} \quad (2)$$

$$\text{eval}(m) = \max_{p \in \text{combin}(m)} \begin{cases} \text{eval}(p) & p \in \mathcal{O}_c \\ \text{avail}(p) & \text{otherwise} \end{cases}, \quad m \in \mathcal{M} \quad (3)$$

$$\text{avail}(v) = \text{eval}(m) + 1, \quad v \in \mathcal{O}_c \text{ driven by } m \quad (4)$$

Four properties follow, and each one is load-bearing.

(P1) Boolean depth is free. Equation (2) takes a maximum, never an increment. An arbitrarily deep cone of AND gates evaluates inside one major stage, exactly as in stock GEM. The boomerang hierarchy is untouched.

(P2) Clocked macro outputs cost nothing. By (1), $\mathcal{O}_s$ is a level-0 leaf. A DSP's P is read from global state like a DFF's Q : what a reader observes this cycle is last cycle's commit, which is the correct register semantics and requires no boundary.

(P3) Macro-to-macro chaining costs nothing. The case split in (3) takes eval , not avail , of an operand that is itself a combinational macro output. Hence a CARRY4 chain of any length — $CO[3] \rightarrow CIN$, the dependency the problem statement names explicitly — resolves inside a single major stage. Ordering within the stage is supplied by emitting one batch per chain link, separated by `__syncthreads()`. Without this case split a 64-bit carry-chain adder would demand 16 major stages, and a wide datapath would exceed the kernel's hard limit of 32.

(P4) Only macro-to-boolean pays. The $+1$ in (4) is charged exactly once, and only where a combinational macro result must cross into the AND-gate domain. It is the minimum price consistent with the buffer separation described in Section 1.

2.3 Stage construction

Let $D = \max_v \text{eval}(v)$ extended over endpoints. Major stage $j \in [0, D]$ is then defined by:

$$\text{endpoints}(j) = \left\{ e : \max_{p \in \text{inputs}(e)} \text{avail}(p) = j \right\} \quad (5)$$

$$\text{macros}(j) = \{ m \in \mathcal{M} : \text{eval}(m) = j \} \quad (6)$$

$$\text{forward}(j) = \{ v : \text{eval}(v) \leq j \wedge \exists c \in \text{consumers}(v), \text{eval}(c) > j \} \quad (7)$$

$\text{forward}(j)$ becomes the staged I/O pin set of stage j and the primary input set of stage $j+1$. Its second clause is the subtle one: a node with $\text{eval}(v) = j$ but $\text{avail}(v) = j+1$ — that is, a combinational macro output produced by stage j 's macro phase — must be forwarded even though it is not readable in the stage that produces it.

Regression guarantee. If $\mathcal{O}_c = \emptyset$ then $D = 0$ by construction, and the entire macro-aware path is bypassed in favour of the unmodified level-split staging. A macro-free netlist is therefore scheduled bit-identically to stock GEM. This is what makes the baseline comparison in Section 5 meaningful rather than circular, and it is asserted directly by unit test `nothing_but_a_combinational_macro_can_raise_a_stage`.

2.4 Measured instantiation

On `synth/tests/macro_smoke.sv` (2 DSP48E2, 4 chained CARRY4, 1 SRLC32E, plus ordinary RTL), the equations yield $D = 1$:

Major stage	Contents	Boomerang depth
0	4 CARRY4 (chained), 1 SRLC32E, 2 DSP48E2, 8 DFF, ordinary cone	7
1	primary outputs consuming macro results	4

The four chained CARRY4s all satisfy $\text{eval} = 0$ by (P3), confirming that the 16-bit carry chain costs no additional grid synchronisation.

On the scaled benchmark `bench/macro_bench.sv` at LANES = 16 (16 DSP48E2, 64 CARRY4 in 16 four-block chains, 16 SRLC32E, 96 macros in all) the equations again yield $D = 1$, and the 64 chained CARRY4s occupy *four batches within one major stage* rather than four stages:

Major stage	Contents	Partitions	Macro batches
0	16 DSP48E2 (endpoint), 64 CARRY4, 16 SRLC32E	2	5
1	boolean cones consuming macro results	1	0

This is (P3) doing the work it exists for. Ordering within a chain is enforced by `--syncthread()` between batches, which is free; only the macro→boolean edge costs a `this_grid().sync()`. Had chaining been charged a major stage per link, this design would have needed five instead of two, and §6 shows that each additional major stage costs $\approx 31 \mu\text{s}$ per simulated cycle — so (P3) is worth roughly $93 \mu\text{s}/\text{cycle}$ on a design this shape.

3 VRAM Layout Architecture

3.1 Two disjoint regions

Macro state is split by lifetime, not by macro:

1. **Bit-addressed outputs** live in the partition’s write-out block, interleaved with ordinary boolean write-outs so that the existing commit path carries them to global state with no separate store.
2. **Word-addressed internal state** — a DSP’s *PREG*, an SRLC32E’s 32-bit shift register — lives in a dedicated `UVec<u64>` region, `macro_word_state`.

3.2 Write-out block ordering

Within a partition the write-out region is ordered

$$\underbrace{\text{normal}}_{n_w} \parallel \underbrace{\text{macro}}_{n_m} \parallel \underbrace{\text{duplicate}}_{n_d} \parallel \underbrace{\text{SRAM}}_{n_s}$$

with SRAM deliberately last so its pre-existing offset arithmetic is untouched and — less obviously — with the macro block *below* the duplicates rather than between them and the SRAMs. That ordering is a correctness requirement rather than a layout preference; §6.5 gives the defect that established it.

Macro instance i receives

$$n_{\text{slots}}(i) = \left\lceil \frac{|\text{outputs}(i)|}{32} \right\rceil \quad \text{contiguous u32 slots,}$$

so bit b of that macro lands at global state bit $32 \cdot \text{macro_start}_i + b$. Two consequences fall out of the allocation rather than from a convention: every macro’s output field is contiguous, and its base is 32-bit aligned. Because each macro owns whole words, no two macros can contend for the same u32, and the device-side result scatter is a plain read-modify-write requiring *no atomics*.

3.3 Word-state region and 64-bit alignment

$$\text{state_words}(k) = \begin{cases} 1 & k = \text{DSP48E2} \quad (P[47:0]) \\ 1 & k = \text{SRLC32E} \quad (32\text{-bit SR}) \\ 0 & k = \text{CARRY4} \quad (\text{stateless}) \end{cases}$$

The region is declared as `UVec<u64>`. The element type supplies 8-byte alignment and `cudaMalloc` supplies a 256-byte aligned base, so the problem statement’s 64-bit alignment requirement holds *structurally* rather than by runtime assertion. Instances are grouped by kind and each kind begins on a warp boundary, so a type-homogeneous batch reads a contiguous, segment-aligned span — the condition for coalesced access.

Measured layout (`macro_smoke`).

Quantity	Value
Macro instances	7
Word-state region	$64 \times \text{u64} = \mathbf{512 \text{ bytes}}$
Alignment	64-bit (verified)
Descriptor words	458
Macro write-out slots	9 (2 DSP $\times$ 2, 4 CARRY4 $\times$ 1, 1 SRL $\times$ 1)

The 458-word descriptor total decomposes exactly as $2(5+123+48) + (5+7+2) + 4(5+10+8) = 352 + 14 + 92 = 458$, where each descriptor carries a 5-word header followed by input and output bit positions. The exact agreement is used as a host-side structural check.

3.4 Script-side indirection

A batch record in the block script references macros by index into the layout’s slot table rather than inlining descriptors:

```
32-bit boomerang stage index this batch runs AFTER
32-bit macro kind code
32-bit count
count x 32-bit slot index into the descriptor CSR
```

A DSP descriptor alone is 176 words; inlining would dwarf the script it lives in. Measured effect on `macro.smoke`: the macro-preserving script is **69,652 words** against the shredded baseline’s **94,720** — a 26 % reduction, the direct instruction-memory dividend of not shredding.

4 Occupancy and Resource Profile

Measured with `cuobjdump -res-usage` on the shipped `sm_86` binary, and with a device query for the machine parameters.

Kernel <code>simulate_v1_noninteractive_simple_scan</code>	
Registers / thread	86 and 105 (ceiling 128)
Stack frame	0 bytes (both variants)
Local memory	0 bytes — no spilling (both variants)
Shared memory	3,328 bytes
Constant bank 0	440 / 408 bytes
Device — NVIDIA RTX 3050 6 GB Laptop, <code>sm_86</code>	
Streaming multiprocessors	20
Registers per SM	65,536
Register allocation granularity	256 per warp
Blocks per SM at 86 reg/thread	2
Blocks per SM at 105 reg/thread	2
Cooperative grid ceiling	$2 \times 20 = \mathbf{40}$ blocks
<code>num_blocks</code> used	40

The fatbin carries two compiled variants of the kernel, at 86 and 105 registers per thread. Which one the driver loads is not determined here, and it does not need to be: both round to the same occupancy. At 256 threads per block, a warp allocates $\lceil 86 \cdot 32 / 256 \rceil \cdot 256 = 2816$ registers, a block $8 \times 2816 = 22,528$, and $\lceil 65536 / 22528 \rceil = 2$ blocks per SM. The 105-register variant gives 2 as well. Every conclusion below is therefore insensitive to the choice.

The 3-blocks/SM threshold, and how narrowly it is missed. Three resident blocks would need $\lceil 65536 / 3 \rceil = 21,845$ registers per block, i.e. 2730 per warp, which the 256-register granularity rounds down to 2560 — exactly **80** registers per thread. At 86 the kernel misses the threshold by six registers per thread, or 512 registers per SM out of 65,536: under one percent. Capping at `-maxrregcount=80` would raise the cooperative grid ceiling from 40 to $3 \times 20 = 60$ blocks, a fifty percent increase in resident parallelism.

That lever is recorded rather than exercised, and §6.2 explains why it would change nothing on the present benchmark: at `LANES = 16` the macro-preserving flow produces three partitions, so it occupies at most two blocks. Raising a ceiling of 40 to 60 cannot help a workload that is using two. The measurement identifies the lever; a design with enough partitions to press against the ceiling is what would make it pay.

4.1 Why the ceiling is a design boundary

`build.rs` pins `-maxrregcount=128`. At 256 threads per block this admits exactly 2 blocks/SM, giving a cooperative grid ceiling of 40 — which is *identical* to `usage.md`’s recommended $2 \times$ SM count. There is no headroom: the advisory figure and the hardware limit are the same number.

The occupancy consequence is therefore not a launch failure. Because `-maxrregcount` caps allocation rather than permitting growth, additional register pressure in the macro phase does not fail to launch — it **spills to local memory and silently loses throughput**. Launch success proves nothing about occupancy health. The metric that does is

```
l1tex__t_sectors_pipe_lsu_mem_local_op_ld.sum
```

under Nsight Compute — or, without profiling privileges, `cuobjdump -res-usage`, which reports the same fact statically as `LOCAL` and `STACK`. Both variants of the shipped kernel report `STACK:0 LOCAL:0`, so the macro phase is inside budget with 22 registers of headroom against the 128 cap. That margin is the reason the correctness fixes could be made without measuring occupancy after each one — but it is not licence: richer DSP modes or chain fusion should be assumed to cross it and must be re-measured, not estimated.

It is worth recording that this figure improved as a side effect of a correctness fix rather than a tuning pass. Moving the `data_inv` correction ahead of the macro phase (§6.5) replaced two XOR operations with one and removed a value that had been live across the entire phase: registers fell from 125 to 86, still with no spilling.

Design consequence. This boundary is why the correctness fixes described in Section 5 were implemented *host-side*, in the scheduler and memory formatter, rather than in `csrc/kernel_v1.impl.cuh`. Every one of them adds zero registers.

5 Dual-Oracle Verification Methodology

The problem statement supplies no golden reference model. We therefore construct two independent oracles and apply each only where it has authority. The central discipline is that **an oracle must share no code with the thing it validates**.

5.1 The script alignment invariant

Injecting variable-length macro batches into GEM’s VLIW-style script stream uncovered an undocumented structural constraint.

The boomerang sections are not read word by word. They are fetched with vector loads:

```
struct alignas(8) VectorRead2 { u32 c1, c2; };
struct alignas(16) VectorRead4 { u32 c1, c2, c3, c4; };
t4_1.read(((const VectorRead4 *) (script + script_pi)) + threadIdx.x);
```

The cast to `VectorRead4*` requires `script_pi` to be a multiple of four `u32` words. Stock GEM satisfies this without ever stating it: the metadata block is 256 words, each global-read round is 256×2 , and each boomerang stage is $256 \times 4 \times 5$. Every constituent is a multiple of four, so the invariant holds by construction and no code asserts it.

The macro section is the first variable-length element ever added to the stream. A batch record is $3 + \text{count}$ words, so *any batch with an even count breaks the alignment for every partition that follows it*:

Macro section contents	Batches	Words	$\equiv 0 \bmod 4$?
4 CARRY4 + 1 SRLC32E, all count = 1	5	20	yes
+ 1 DSP48E2 batch, count = 2	6	25	no

The failure mode is instructive. It is not a wrong answer and not an illegal address; it is `CUDA_ERROR_MISALIGNED_ADDRESS`, raised at the cooperative launch’s synchronisation point, naming neither the script nor the macro that caused it. It was identified by the distinction between *misaligned* and *illegal*: an illegal address would have implicated descriptor indexing, whereas misalignment can only come from a widened load. That the fault was identical at 2 blocks and at 40 excluded any block-count dependence.

The fix is host-side, in the script emitter, and costs no registers:

```
while script.len() % 4 != 0 { script.push(0); }
```

We record this as a general design rule for the architecture: **any variable-length section spliced into a GPU instruction stream consumed by widened loads must preserve the widest load’s alignment**, and that requirement should be asserted at emission rather than discovered at launch.

5.2 What each oracle can and cannot establish

The three oracles are not interchangeable, and the distinction is not academic — each has produced a false or incomplete verdict in this project.

Oracle	Establishes	Cannot establish
CPU reference (<code>--check-with-cpu</code>)	<i>Implementation fidelity</i> : the GPU kernel computes what GEM’s own CPU model computes, from the same script	<i>Design correctness</i> . Both sides consume the same mapped script, so a synthesis or mapping defect appears identically in both and passes
Shredded baseline VCD	<i>Cross-implementation agreement</i> : the macro path and a macro-free path that shares no code with it agree	Correctness of the baseline itself. It inherits every synthesis and mapping defect of the shredded flow
Design-level Python testbench	<i>Semantic correctness</i> against IEEE 1800–2012 SystemVerilog intent, independent of both flows	Nothing — but only once calibrated against a known-good signal

5.3 Oracle 1 — CPU reference, for the boolean path

GEM ships a CPU simulator, invoked with `--check-with-cpu`. It is a legitimate oracle for the shredded baseline, which contains only AND-inverter logic and flip-flops, and the baseline passes it. Its authority is bounded, however: it compares two implementations of the *same script*, so it validates that the GPU is faithful to the CPU model, not that either is faithful to the RTL.

It is *not* an oracle for the macro path, and treating it as one produced the most instructive failure of this project. At one point the native run reported **sanity test passed** while every macro-driven output in the VCD was frozen at its initial value for all 2001 cycles. The CPU reference does not model macro semantics, so it agreed with the GPU about bits that neither was computing. Two errors cancelled, and a green result concealed a completely dead datapath.

5.4 Oracle 2 — shredded-baseline VCD, for the macro path

The macro-preserving and shredded builds are the *same RTL* driven by the *same stimulus*. The shredded build is independently validated by Oracle 1. Their output waveforms must therefore agree at every timestamp on every port. Crucially, the shredded path exercises no macro code whatsoever: the two share no implementation, so agreement is evidence and not tautology.

`bench/compare_vcd.py` performs this comparison by port name (VCD identifier codes are per-file and meaningless across files), reporting ports present in only one file separately rather than silently ignoring them. The comparator itself is validated in both directions: it passes on identical input and detects a deliberate three-bit mutation.

This oracle immediately localised what Oracle 1 had missed — 110 of 123 ports differing, partitioned cleanly by macro kind — and each subsequent fix moved a specific port group from differing to matching.

Establishing that the baseline is trustworthy. A cross-implementation oracle is only as good as the reference implementation, so the baseline was itself validated against a third, independent simulator: GEM’s `naive_sim`, which evaluates the gate-level netlist directly on the CPU with no script, no partitioning and no boomerang scheduling. It shares essentially no code with the GPU path.

Comparison (400 cycles, 123 ports)	Ports differing
<code>naive_sim</code> vs shredded GPU	0 / 123
<code>naive_sim</code> vs macro-preserving GPU	108 / 123

The shredded GPU run is bit-exact against netlist-level ground truth on every port. The baseline is therefore sound, and a disagreement reported by Oracle 2 is attributable to the macro path.

A retracted inference, recorded deliberately. An earlier draft of this report asserted that the baseline was defective on `sum[16]`, reasoning that the carry-out of a 16-bit addition cannot be constant under pseudo-random stimulus, whereas the shredded waveform records one value change in 2001 cycles. `naive_sim` reproduces that same constant from the netlist. The netlist behaviour is what it is; the faulty step was reasoning from RTL intent *across* a synthesis boundary that had not been verified. We record the retraction rather than silently deleting it, because it is the same class of error the methodology exists to catch: a confident inference that no measurement supported.

5.5 Oracle 3 — design-level Python testbench

`bench/golden_model.py` evaluates `macro_smoke.sv`’s RTL semantics directly in Python from the same input VCD, sharing no code with either synthesis flow or with the simulator. It is the only oracle that validates against *design intent* rather than against another implementation, and it is what attributes a disagreement to one side or the other.

Status: authored, not yet calibrated. The model does not currently reproduce `scratch` (`cnt  $\oplus$  {x[3:0], y[3:0]}`), a macro-free signal that Oracle 1 has already validated. A model that disagrees with a known-good signal is wrong about that signal, so its verdicts on the macro-driven ports are withheld. Calibration was attempted over input-sample offsets ± 2 cycles, both nibble orderings, and all 256 counter phases; the best fit was 14 of 30 cycles, indistinguishable from chance. The mismatch is therefore structural — in how GEM’s cycle indexing maps onto the generated stimulus — rather than a parameter to be swept. **Calibrating on scratch is the prerequisite for every remaining correctness claim in Section 6.**

We state this rather than presenting an uncalibrated model as evidence: the methodological point of this section is that an unvalidated oracle is worse than none, and the same standard applies to our own.

5.6 Oracle 4 — cross-language macro models

For the primitive models themselves (Deliverable C) the oracle is two independent transcriptions of the problem statement’s equations, in C++ (`csrc/macros.cuh`, compiled both as host code and as `__device__` code) and in Python (`csrc/tests/reference_model.py`), diffed bit for bit over an identical vector set:

CARRY4	1,024 cases — exhaustive ($16 \times 16 \times 2 \times 2$)
DSP48E2	1,648 cases (8 directed + 400 random $\times$ 4 accumulate cycles)
SRLC32E	11,520 reads + 360 clock edges
Total	14,552 vectors, 0 disagreements

Because the CUDA kernel and the host golden model compile from one source, a separate CPU reference cannot drift from the GPU path — the verification argument rests on them being the same code, checked against a different-language restatement of the specification.

6 Results

6.1 Front-end: the cost of shredding

Identical RTL, two synthesis flows, on both benchmarks:

Design	Flow	AIG cells	DFFs	Macros	Partitions
<code>macro_smoke</code>	shredded	9,922	133	0	1
	preserved	95	8	7	1
<code>macro_bench</code> LANES = 16	shredded	85,444	1,600	0	6
	preserved	5,754	768	96	2 + 1

104 $\times$ and 14.8 $\times$ reductions in AIG cell count. That shredding a handful of word-level primitives inflates the graph by one to two orders of magnitude is the architectural claim, and these are the numbers behind it.

The DFF counts move too, and for a reason worth naming: a DSP48E2’s *P* register and an SRLC32E’s shift register become *macro word state* rather than boolean flip-flops, so they leave the boomerang’s endpoint set entirely.

6.2 Throughput

`macro_bench` at LANES = 16. Same GPU, same stimulus, same `num_blocks` = 40, same 400 cycles, same binary; five repetitions per side, median reported. Timing is GEM’s own `simulation` span, which brackets the kernel launches and excludes parsing, mapping and VCD writing.

Configuration	Median	Cycles/s	Script/cycle	Per cycle
Shredded baseline	18.65 ms	21,445	2,060.0 KiB	46.6 μ s
Native macros	31.09 ms	12,866	454.5 KiB	77.7 μ s
Ratio		0.60$\times$	4.53$\times$ smaller	+31.1 μ s

The optimisation succeeds on its own metric and loses on wall clock. GEM re-reads its entire instruction script from global memory on every simulated cycle, so script size is the per-cycle DRAM traffic floor and is exactly the quantity this work attacks. It fell $4.53\times$, from 527,360 to 116,340 u32 words. Runtime nevertheless rose by $31.1\ \mu\text{s}$ per cycle. Both statements are measurements; neither is qualified.

Therefore the workload is not bandwidth-bound. This is the load-bearing inference, and it is falsifiable rather than rhetorical. The native configuration reads 1.64 MB *less* every cycle. Were the run limited by script bandwidth, its script-read component would have fallen to 22% of the baseline’s and total runtime would have fallen with it. It rose. Expressing the same data as achieved script-read rates makes the gap plain:

Configuration	Script traffic	Elapsed	Achieved rate
Shredded baseline	843.8 MB	18.65 ms	45.2 GB/s
Native macros	186.1 MB	31.09 ms	6.0 GB/s

At 6.0 GB/s the native path is nowhere near the memory system’s limit. It is waiting on something else.

Two candidate causes, and they are separable. (i) *The additional major stage.* Macro-depth staging places boolean consumers of a macro result one major stage above their producer, so the native path executes two `this_grid().sync()` per cycle against the baseline’s one. A grid barrier of $\approx 31\ \mu\text{s}$ is the right order for this part.

(ii) *Partition-count collapse.* GEM’s unit of parallelism is the *partition*, and the front-end table above shows the native flow needs three where the baseline needs six — and splits them across two sequential stages, so at most two blocks are resident at any instant. With `num_blocks = 40` both configurations badly under-fill the device, but the native one under-fills it by a further factor of three.

Cause (ii) is the more interesting of the two, because it is not an implementation defect but a structural tension in the approach: **fewer cells means fewer partitions means less parallelism**. The script saving is real, and it only pays once the design is large enough that both flows still produce many partitions. This reframes what a larger benchmark tests — not whether the script shrinks, which is settled, but whether partition count survives the shrinking.

Section 6.3 reports the Nsight Compute measurement that discriminates between (i) and (ii).

What this result is not. It is not evidence that native macro evaluation is a bad idea, and it is not a speedup. It is a measurement of where the crossover is not yet. The honest summary is that at LANES = 16 on an RTX 3050 Laptop, the design is too small for a $4.53\times$ bandwidth saving to outweigh one extra grid barrier and a threefold loss of occupancy.

6.3 Kernel profile

Metric	Shredded	Native
<code>dram__throughput</code> (% of peak)	[PENDING]	[PENDING]
<code>sm__warps_active</code> (% of peak)	[PENDING]	[PENDING]
<code>smsp__thread_inst_executed</code> (of 32)	[PENDING]	[PENDING]
Local memory / stack (static)	0 / 0	0 / 0

The last row is measured and the other three are not, because Nsight Compute requires elevated access to the GPU performance counters (`ERR_NVGPUCTRPERM`) which was unavailable on the measurement machine. The spill check does not depend on it: `cuobjdump -res-usage` reports `STACK` and `LOCAL` statically from the compiled binary, and both read zero (§4). That is the row on which the others depend, so the analysis in §6.2 stands.

Three readings are decisive. A baseline `dram_throughput` in single digits confirms directly that the workload was never bandwidth-bound at this scale, which is currently inferred from timing alone. A native `sm_warps_active` materially below the baseline’s implicates cause (ii) over cause (i). And the local-memory counters must read *zero*: `-maxrregcount caps` registers rather than failing to launch, so excess pressure spills silently — a successful launch proves nothing, and every conclusion in §6.2 is unsafe if these are non-zero.

6.4 Verification status

All results below are measured on an RTX 3050 Laptop (`sm.86`) with identical stimulus and identical `num_blocks` on both sides.

Rung	What it isolates	Result	Comparisons
Macro models	PS equations, C++ vs Python	Pass	14,552 vectors
<i>macro_bench, LANES = 16 — 82 ports × 400 cycles</i>			
1. Shredded GPU vs <code>naive_sim</code>	the ordinary boolean path	Pass	32,800
2. Design equivalence (CPU only)	macro-preserving synthesis	Pass	32,800
3. Macro GPU vs <code>naive_sim</code>	the CUDA kernel	Pass	32,800
4. Compiled script vs <code>naive_sim</code>	<code>pe.rs</code> / <code>flatten.rs</code>	Pass	32,800
<i>macro_smoke regression — 123 ports × 400 cycles</i>			
2. Design equivalence (CPU only)	macro-preserving synthesis	Pass	49,200
4. Compiled script vs <code>naive_sim</code>	<code>pe.rs</code> / <code>flatten.rs</code>	Pass	49,200

On `macro_bench` the macro-preserving path evaluates 16 DSP48E2, 64 CARRY4 — including 16 four-deep `C0[3]→CIN` chains — and 16 SRLC32E natively on the GPU ALU, and reproduces netlist-level ground truth at every port and every timestamp.

One dependency in that design exists purely as a regression test. `flags[1] = (sum_1 > 17’d100)` is ordinary AND-gate logic consuming a word-level CARRY4 result: precisely the macro→boolean edge that was silently broken before the staging equations of §2 existed. It is exercised sixteen times per cycle.

Rung 4 is what made the remaining defects findable. Once `flatten_test` could execute the compiled script on the CPU, a rung-3/rung-4 disagreement localised the fault to the kernel and an agreement localised it to the host compiler — where it could be reproduced and bisected in seconds instead of through GPU round-trips. Both of the defects in §6.5 were found that way.

The middle rung is what made the remaining defects findable. Once `flatten_test` could execute the compiled script on the CPU, comparing it against the GPU isolated the kernel — which agreed with it exactly, so every surviving error was in the host compiler, where it could be reproduced and fixed in seconds rather than through GPU round-trips.

6.5 Defects the ladder exposed

Every one was silent: correct-looking execution, wrong arithmetic, no diagnostic anywhere.

Write-out polarity. A write-out slot stores the value of `pin_iv`, which for an inverted pin is the complement of the AND-gate cover’s raw result; the `data_inv` table restores it at commit. The macro phase ran *before* that correction and therefore read every inverted operand backwards — observed as an exact whole-vector inversion on a CARRY4’s 0 outputs. The correction now applies to `shared_writeouts` in place, ahead of the macro phase; macro results are written afterwards and carry `data_inv = 0`, so they are correctly left alone.

Constant-one operands. In the AIG’s `iv` encoding both constants live on aigpin 0: `iv = 0` is false and `iv = 1` is *true*. The formatter mapped both to the “no bit” sentinel, which reads as zero. A DSP instantiated with `.USE_PREADD(1'b1)` therefore never applied its pre-adder, and one with `.MODE(2'd1)` silently became mode 0 — bypass instead of multiply. One line, two unrelated-looking symptoms, no error at any point. `macro_layout::CONST.ONE` is now a distinct sentinel honoured by the formatter, the CPU executor and `GEM_RD_BIT` alike.

Duplicate write-outs landing in the macro block. The kernel materialises the duplicate write-out words at

$$\text{shared_writeouts}[n_{io} - n_s - n_d + i],$$

locating the block by subtracting from the *top* of the write-out region. Under stock GEM’s layout, `normal||duplicate||SRAM`, that expression evaluates to n_w and is correct by construction. Inserting the macro block between the duplicates and the SRAMs shifts it up by n_m words: every duplicate is then written *inside the macro block*, and the slots the host actually assigned retain whatever the boomerang left in them.

Nothing detects this. All positions remain in range, so the host-side bounds assertion passes; the macro phase subsequently overwrites the clobbered macro words with correct results, so macro *outputs* still verify. Only operands *read* through a duplicate slot are wrong — that is, only pins a macro requires in the opposite polarity from the one the AND-gate cover produced.

The symptom was correspondingly strange. On `macro_bench`, CARRY4 blocks 2 and 3 were exact and blocks 0 and 1 were wrong. Bits 8–15 lie above the reach of the per-lane perturbations, so all sixteen lanes there share one signal in one polarity and need no duplicate; the low blocks need both polarities of x and $x \oplus y$. The error injected at the block-1 boundary then propagated up the carry chain with a per-bit halving decay, which reads convincingly as an arithmetic bug in the CARRY4 model — where nothing was wrong.

The fix reorders the region to `normal||macro||duplicate||SRAM`, restoring the invariant the kernel already assumes. Two lines in `flatten.rs` and **no kernel change**, which matters directly: teaching the kernel about n_m would have cost a metadata word and a live value against the register budget of §4.

6.6 What the harness must guarantee

Two of this project’s longest debugging episodes were not defects in the simulator at all. They are worth recording because both are structural traps that any GPU-simulator project can fall into, and both are now prevented by construction rather than by remembering.

A .gemparts file is a build product, not an input. It serialises each partition’s stages *and its macro batches*, so it encodes a snapshot of the scheduler in `pe.rs`. Changing the scheduler and reusing the file causes the simulator to faithfully execute the old schedule. One such run presented sixteen DSP48E2 instances that were endpoints, held state slots and committed every cycle — and were never once scheduled into a batch, so `gem_dsp48e2` was never called. Nothing reports that; the accumulators simply never move. `bench/verify_ladder.sh` now re-maps both flows itself.

A successful build is not proof the binary is current. `Cargo.toml` declares `required-features = ["cuda"]` on `cuda_test`. `Cargo` *skips* a binary whose required features are absent — silently, with exit status 0. A plain `cargo build --release` therefore relinks `naive_sim`, `flatten_test` and `cut_map_interactive`, leaves `cuda_test` untouched, and reports success. Successive fixes were then evaluated against a stale kernel binary, producing a `rung-3` failure whose mismatch counts were *identical to the digit* across 32,800 comparisons on twelve ports — which is what finally identified the cause, since no code change can leave a wrong answer bit-for-bit unchanged. The ladder now builds with `--features cuda` and additionally refuses to run if any file under `src/`, `csrc/`, `build.rs` or `Cargo.toml` is newer than the binary.

The general principle is the one this whole section argues: a verification harness that can report a result it has not actually established is worse than no harness, because it converts an absent measurement into a confident wrong one.

7 Measurement Provenance

Figure	Source
Cell counts	<code>synth/check_macros.sh</code> on both flows
Partition counts	<code>cut_map_interactive</code> “after merging” log
Script size	<code>flatten.rs</code> “Built script for” log line
Throughput, script size	<code>bench/throughput.sh</code> , 5 reps, median
Kernel profile	<code>bench/profile.sh</code> (Nsight Compute)
Ladder results	<code>bench/verify_ladder.sh</code>
Register / local memory	<code>cuobjdump -res-usage</code> on <code>libgemcu.a</code>
SM count, blocks/SM	<code>bench/occupancy_probe.cu</code>
Word-state size, descriptors	<code>cut_map_interactive</code> log
Kernel time	<code>cuda_test</code> internal timer (excludes VCD parsing)
Port agreement	<code>bench/compare_vcd.py</code>
Macro model vectors	<code>csrc/tests/verify_macros.sh</code>

Kernel time is GEM’s own instrumented figure and excludes VCD parsing, which `usage.md` warns can dominate wall-clock time. Baseline and native are measured with the identical metric on the identical device.